Advanced Micrograph Computer Vision Handout: Decoupling Polymer Reticulation States via 2D-CNN Latent Spaces

Llinersy Uranga Piña, LPPI - Cergy-Paris Université

June 2026

Contents

1 Introduction: Automated Structural Fingerprinting

In materials science and microscopy characterization (e.g., examining Scanning Electron Microscopy (SEM) or Transmission Electron Microscopy (TEM) micrographs), quantifying spatial reticulationthe degree of cross-linking, crystalline grid formation, or fiber network interconnectionis challenging. Manual inspection introduces human bias, while simple pixel-density thresholding fails when images contain complex aberrations, focus drift, or electronic detector noise.

The computational architecture implemented in `module5-CNN-images` uses a 2D Convolutional Neural Network (CNN) to automatically extract geometric fingerprints directly from raw micrographs. Rather than functioning as an unverifiable "black-box" predictor, this workbook extracts internal convolutional activations and projects intermediate dense layers into low-dimensional latent spaces. This approach provides experimentalists with clear, numerical verification of structural differences.

2 Micrograph Synthesis: Modeling Physical Morphologies

To train and test the neural network, the pipeline generates synthetic image matrices (64×64 pixels) that model distinct physical structures:

2.1 1. Class 0: Low Reticulation (Amorphous Matrix)

This category simulates an unlinked or amorphous material state. The code distributes isolated coordinates randomly across the matrix and applies a light localized Gaussian blur ($\sigma = 1.0$):

```
img_low[p[0], p[1]] = 1
img_low = filters.gaussian(img_low, sigma=1) + np.random.normal(0,
    noise_level, ...)
```

This represents isolated structural points, point defects, or amorphous material configurations mixed with standard instrumental noise.

2.2 2. Class 1: High Reticulation (Interconnected Crystalline Grid)

This category simulates a highly cross-linked, crystalline, or self-assembled framework. The code draws systematic intersecting continuous lines spaced 8 pixels apart, followed by a wider blurring pass ($\sigma = 1.5$) to mimic realistic microscope electron-beam aberrations and variations in sample thickness:

```
for i in range(0, img_size, 8):
    img_high[i, :] = 1
    img_high[:, i] = 1
img_high = filters.gaussian(img_high, sigma=1.5) + np.random.normal(0,
    noise_level, ...)
```

This pattern provides the complex grid networks used to test the network's spatial feature extraction capabilities.

3 2D Convolutional Networks & Latent Deconvolution

3.1 1. Spatial Feature Extraction via 2D Kernels

Unlike standard sequential signal operations, a 2D convolutional layer slides a small spatial grida kernelacross the image matrix. The architecture uses 3×3 filters:

```
layers.Conv2D(16, 3, activation='relu', padding='same')(inputs)
```

Mathematically, the layer calculates a localized dot product at each coordinate (x, y) within the image:

$$O(x,y) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} I(x+m, y+n) \cdot K(m,n) + b \tag{1}$$

where I represents input pixel intensity, K is the kernel weight grid, and b is a scalar bias term. These layers function as automated edge, junction, and curvature detectors, capturing structural details without requiring manual input from the researcher.

3.2 2. Translation Invariance via Max-Pooling

To ensure the model recognizes a fiber network regardless of where it appears in the frame, the code follows each convolution with a Max-Pooling step:

```
layers.MaxPooling2D(2)(x)
```

This layer slides a 2×2 window across the feature map and retains only the maximum value within that neighborhood:

$$P(i,j) = \max\left(O(2i,2j), O(2i+1,2j), O(2i,2j+1), O(2i+1,2j+1)\right) \tag{2}$$

This halves both vertical and horizontal dimensions, reducing computational overhead and ensuring the model remains sensitive to structural features even if the sample shifts slightly during imaging.

3.3 3. Unlocking the Latent Core Space

Before reaching the final binary decision output (a single sigmoid neuron), the extracted structural features are passed through a highly compressed 4-dimensional dense layer:

```
latent_features = layers.Dense(4, activation='relu', name='
    latent_dense_features')(x)
```

This layer forces the network to compress the entire spatial image down to just four numerical variables. These variables function as automated descriptors that mathematically capture the core physical differences between the material classes.

4 Interpreting Latent Feature Space Metrics

To evaluate the model's inner logic, the pipeline isolates this intermediate dense layer using a sub-model mapping and plots the results as a boxplot distribution:

```
feature_extractor = Model(inputs=model.input, outputs=model.get_layer('
    latent_dense_features').output)
```

4.1 1. Mapping Feature Separability

The resulting boxplot separates the four internal descriptors along the horizontal axis and plots their respective activation values vertically. Descriptors that display large, distinct gaps between Class 0 and Class 1 serve as primary indicators for structural evaluation.

4.2 2. Physical Correlations (Lab Task 3 Insight)

In this architecture, the features displaying the widest separation gaps correlate directly with total edge density per unit area. Because the reticulated grid network features continuous intersecting walls, it contains a much higher density of sharp intensity gradients compared to the scattered dots of the amorphous phase. The convolutional kernels function as localized edge detectors, mapping this physical discrepancy directly to high activation scores in the dense layer.

5 Robustness Testing & Scaling Constraints

5.1 1. Image Noise Thresholds (Lab Task 1 Insight)

When simulated electronic detector noise is increased from $\sigma = 0.1$ to an extreme $\sigma = 0.5$, the raw images become heavily degraded. While the convolutional kernels can filter out some random variation by averaging nearby pixels, the overall validation accuracy typically drops by 8% to 15%.

In the feature space plot, the activation distributions begin to overlap. This overlap occurs because high-frequency random noise occasionally forms accidental clusters that mimic the key features of the reticulated grid lines, causing the model to misclassify those regions.

5.2 2. Resolving Power and Spatial Limits (Lab Task 2 Insight)

Downscaling the input image frame from 64×64 to 32×32 pixels reduces the line spacing to less than 4 pixels. After applying the Gaussian blur ($\sigma = 1.5$), individual continuous lines bleed into each other, destroying the spatial gaps that define the network structure. At this point, the features look identical to random noise clusters. As a result, the network cannot isolate distinct shapes, and its performance drops close to random guessing (50%), marking the absolute resolution limit for this architecture.

5.3 3. Cross-Domain Transfer Learning (Lab Task 4 Insight)

- **Feasibility:** A model trained on Covalent Organic Frameworks (COFs) can be repurposed to analyze Metal-Organic Framework (MOF) porosity via transfer learning. Both materials share a similar topological layout defined by dense structural walls surrounding regularly spaced pore voids.
- **Scale-Shift Constraints:** However, if the physical scale (nanometers per pixel) changes significantly, the model will likely fail. Convolutional networks are highly sensitive to the absolute size of features in pixels. If a COF pore spans 20 pixels but a MOF pore spans only 3 pixels due to a change in microscope magnification, the original convolutional filter will slide right past the smaller pores without registering them. Researchers must resize new datasets to match the original training scale or use a multi-scale network architecture.